

Name: Dahiwal Gajanan Manikrao
Roll No: EN24107029
Class: SY - AI \& DS
Subject: Python for Visual AI

Assignment 4:

An AgriTech startup wants to predict whether a farmer should grow Rice, Wheat, or Maize based on soil and climate conditions using crop recommendation dataset.

1. Understand features: N, P, K, Temperature, Humidity, Rainfall
 2. Define input—output mapping
 3. Identify ML problem type
 4. Perform exploratory data analysis (EDA)
-
-

Step 1: Importing the necessary python libraries

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import math
from collections import Counter
```

Step 2: Loading the dataset into pandas dataframe

```
In [ ]: df=pd.read_csv("Downloads/Crop_recommendation.csv")
df.head()
```

```
Out[ ]:
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Step 3: Basic Data Analysis

We analyze the dataset using:

- `info()`: Structure of dataset
- `describe()`: Statistical summary
- `shape`: Number of rows and columns
- `dtypes`: Data types of features
- Missing Values Check

```
In [ ]: df.info()
df.describe()
df.shape
df.dtypes
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   N                2200 non-null   int64
1   P                2200 non-null   int64
2   K                2200 non-null   int64
3   temperature      2200 non-null   float64
4   humidity          2200 non-null   float64
5   ph               2200 non-null   float64
6   rainfall          2200 non-null   float64
7   label            2200 non-null   object
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

```
Out[ ]: N                0
P                0
K                0
temperature      0
humidity         0
ph               0
rainfall         0
label            0
dtype: int64
```

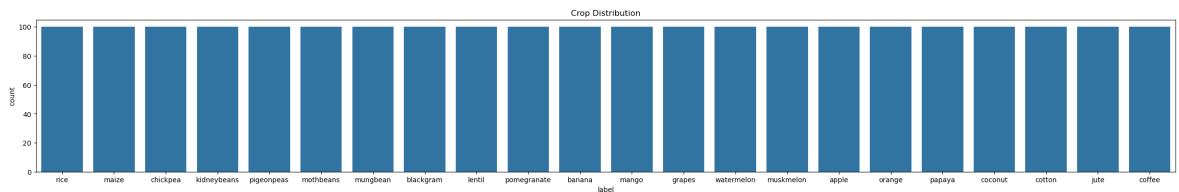
Step 4 : Crop Distribution

A count plot is used to visualize the distribution of crops in the dataset.

Count Plot → Frequency of each crop label

This helps in identifying class balance in the dataset.

```
In [ ]: plt.figure(figsize=(30,4))
sns.countplot(x="label", data=df)
plt.title("Crop Distribution")
plt.show()
```



Step 5 : Feature Selection

The target variable (label) is removed to create a feature-only dataset.

X = All features except label

```
In [ ]: df1=df.drop("label",axis=1)
```

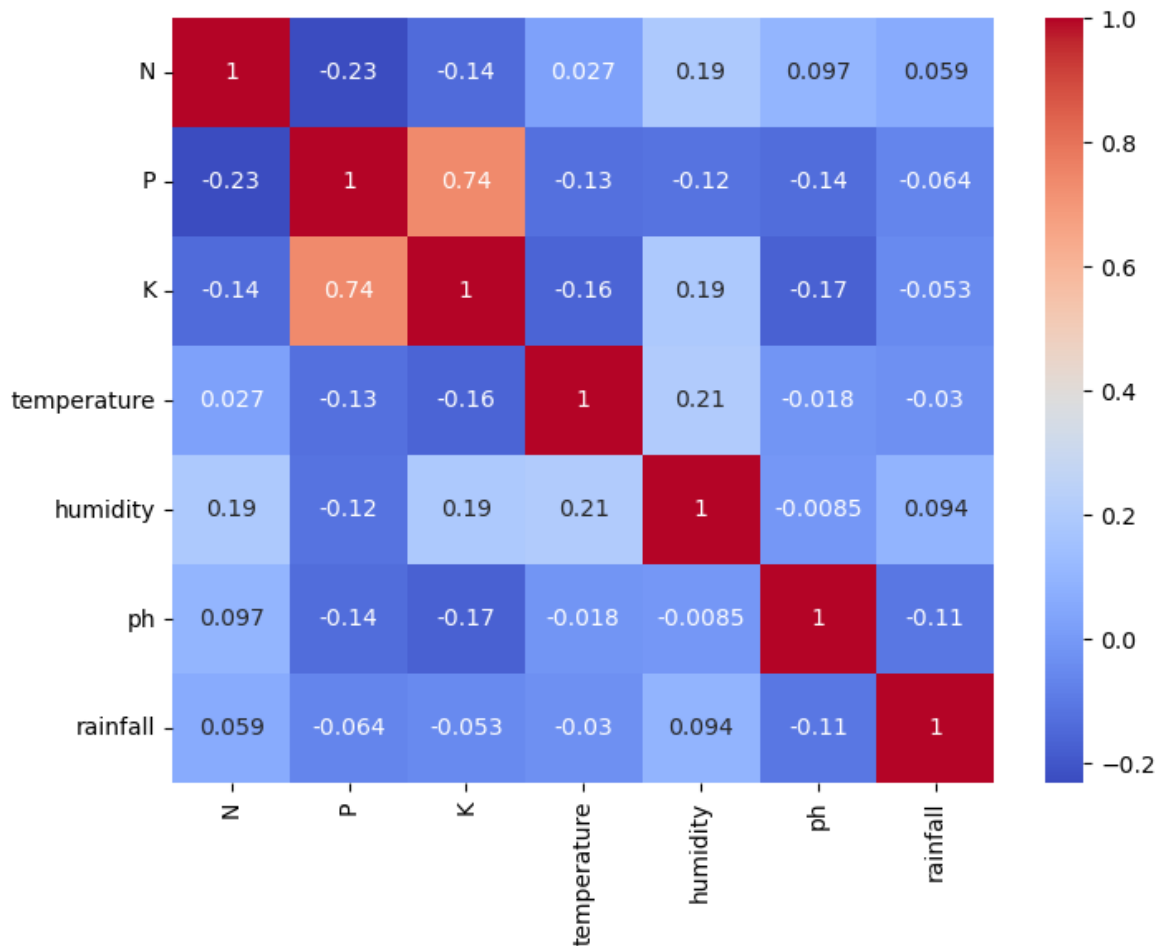
Step 6 : Correlation Analysis

A heatmap is used to analyze relationships between features.

Correlation Matrix = Measure of relationship between variables

This helps in understanding feature dependencies.

```
In [ ]: plt.figure(figsize=(8,6))
sns.heatmap(df1.corr(), annot=True, cmap="coolwarm")
plt.show()
```



Step 7 : Input--Output Mapping

The dataset is split into input features and target variable.

$$X = [N, P, K, \text{temperature}, \text{humidity}, \text{pH}, \text{rainfall}]$$

$$y = \text{Crop Label}$$

This defines the supervised learning setup.

```
In [ ]: x = df[["N","P","K","temperature","humidity","ph","rainfall"]].values.tolist()
        y = df["label"].values.tolist()
```

Step 8 : K-Nearest Neighbors (KNN) Implementation

A manual implementation of KNN is used.

Euclidean Distance Formula:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Working:

1. Calculate distance from test point to all training points
2. Sort distances
3. Select k nearest neighbors
4. Perform majority voting
5. Return predicted class

```
In [ ]: import numpy as np
import math
from collections import Counter

class ManualKNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = X_train.values if hasattr(X_train, "values") else X_train
        self.y_train = y_train.values if hasattr(y_train, "values") else y_train

    def _euclidean_distance(self, p1, p2):
        distance = 0
        for i in range(len(p1)):
            distance += (p1[i] - p2[i]) ** 2
        return math.sqrt(distance)

    def predict(self, X_test):
        X_test = np.array(X_test)
        predictions = [self._predict_single(x) for x in X_test]
        return predictions

    def _predict_single(self, x):
        distances = []

        for i in range(len(self.X_train)):
            dist = self._euclidean_distance(x, self.X_train[i])
            distances.append((dist, self.y_train[i]))

        distances.sort(key=lambda x: x[0])
        k_neighbors = distances[:self.k]
        k_labels = [label for (_, label) in k_neighbors]

        return Counter(k_labels).most_common(1)[0][0]
```

Step 9 : Model Training and Prediction

The KNN model is trained and used for prediction.

$$\text{Model} = \text{KNN}(k = 3)$$

$$\text{Prediction} = f(X_{\text{test}})$$

The model predicts the crop based on given input conditions.

```
In [ ]: model = ManualKNN(k=3)
        model.fit(x,y)

        model.predict([[107,34,32,26,66,177]])
```

```
Out[ ]: ['cotton']
```

Conclusion

The crop recommendation system uses machine learning to predict suitable crops like Rice, Wheat, and Maize based on soil nutrients and climatic conditions. By analyzing features such as N, P, K, temperature, humidity, and rainfall, the system helps in making accurate agricultural decisions. Exploratory Data Analysis helped in understanding data patterns and relationships. Since the problem is a supervised multi-class classification task, models like KNN can effectively classify crops. Overall, this approach improves farming efficiency and supports better crop selection.